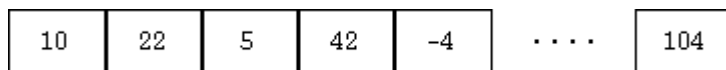


Vektorer

Vi skall nu behandla den första sammansatta datatypen i C++, nämligen *vektorer* (kallas även arrays). En vektor i C++ fungerar precis som i vilket programmeringsspråk som helst, men syntaxen för deklarering och användning skiljer sig förstås lite från andra språk. En vektor är en rad med element av en viss datatyp som finns tätt packade sekventiellt i minnet. En vektor av `int` kan ses som på nedanstående sätt:

Figur 9-2. Schematisk bild över en `int`-vektor



En vektor innehåller alltid ett visst antal element, och vart och ett av dessa element är unikt och kan användas separat från de övriga. Det kan läsas, tilldelas, jämföras o.s.v. Allmänt deklareras en vektor i C++ på följande sätt:

```
datatyp vektornamn [storlek];
```

Man skall alltså ge en storlek på vektorn då den definieras. Några vektordefinitioner finns nedan:

```
int TalFoljd [16];  
double Koordinater [3];  
string Meningar [1024];
```

Den maximala storleken på vektorer är i teorin obegränsad, men i praktiken tar det tillgängliga minne slut i något skede. Storleken bör vara positiv. Storleken bör tolkas som antalet element vektorn kan innehålla. Då en vektor deklareras kan man även direkt initiera dess innehåll. Detta görs genom att lista elementen i ordningsföljd innanför {}. Detta kan endast göras då vektorn deklareras. Exempel:

```
int BitVarden[8] = { 1, 2, 4, 8, 16, 32, 64, 128 };
```

Vektorn `BitVarden` är nu en vektor med åtta element. Det är inte tillåtet att göra följande:

```
int BitVarden[8];  
BitVarden = { 1, 2, 4, 8, 16, 32, 64, 128 };
```

Man kan däremot lämna bort storleken på vektorn och låta kompilatorn fylla i den om man vill. Ovanstående skulle då bli:

```
int BitVarden[] = { 1, 2, 4, 8, 16, 32, 64, 128 };
```

Antalet element i vektorn kan då räknas ut som *totala storleken / storleken på ett element*. Ovan nämnda vektors storlek skulle t.ex. bli:

```
int BitVarden[] = { 1, 2, 4, 8, 16, 32, 64, 128 };  
int Antal = sizeof (BitVarden) / sizeof (int);
```

Indexering av vektorer

För att en vektor skall kunna användas bör man kunna *indexera* de enskilda elementen i vektorn. Med indexering avses att de enskilda elementen kan manipuleras på olika sätt. I detta avseende använder C++ samma syntax som många andra språk. Allmänt ser det ut så här:

```
vektornamn [index];
```

Man ger alltså ett *index*, varvid vektorn ger det element som finns på den givna platsen i vektorn. Observera att vektorer alltid indexeras från 0 i C++! En vektor som är definierad som `int TalFoljd [10];` har 10 element, med index 0 till 9. Att indexera en vektor fel är ett vanligt fel man kan göra i C++. Vad händer om man indexerar fel? Då refererar man minne som inte tillhör vektorn, utan som kan tillhöra någon helt annan variabel som råkar finnas efter den avsedda vektorn. Resultatet är i de flesta fall inte det avsedda. Ett enkelt program som fyller en vektor med kvadrerade tal och skriver ut dem senare finns nedan:

```
#include <iostream>
#include <iomanip>

int main () {
    int Tal [10];

    // iterera och fyll vår vektor
    for ( int Index = 0; Index < 10; Index++ ) {
        Tal [Index] = Index * Index;
    }

    // iterera nu och skriv ut talen
    cout << "Tal   kvadrat" << endl;
    for ( int Index = 0; Index < 10; Index++ ) {
        cout << setw (3) << Index << setw (9) << Tal [Index] << endl;
    }
}
```

Precis som en avrefererad pekare kan en indexerad vektor användas på alla ställen där datatypen normalt kan användas. Programmet ovan ger följande utskrift:

```
% Vectors1
Tal   kvadrat
0           0
1           1
2           4
3           9
4          16
5          25
6          36
7          49
8          64
9          81
%
```

Tilldelning och jämförelse av vektorer

I C++ kan man inte jämföra vektorer med varandra direkt med operatoren `==` av en orsak som visas senare. Istället måste man kontrollera likhet genom att jämföra vektorernas innehåll, d.v.s man måste jämföra varje element i taget, och på så vis komma fram till om likhet gäller eller inte. Samma gäller för tilldelning. Man kan inte direkt tilldela en vektor värdet av en annan vektor, utan man måste

tilldela elementen ett i taget. Detta förfarande kan verka en aning klumpigt.

[Förutgående](#)

Avancerade datatyper

[Hem](#)

[Upp](#)

[Nästa](#)

Vektorer som pekare